SKILLLOOP

A Campus-Based Peer-to-Peer Skill Exchange Platform

Project Report

Submitted in partial fulfilment of the requirements for the course: [COURSE / SUBJECT NAME]

TEAM MEMBER	ROLL NUMBER
Aman	[ENTER ROLL NUMBER]
Iksha	[ENTER ROLL NUMBER]

Department: [DEPARTMENT NAME]

Institution: [COLLEGE / UNIVERSITY NAME]

Faculty Guide: [FACULTY NAME]

Academic Year: 2026–27

Submission date: [ENTER DATE]

01 — PROJECT OVERVIEW

Abstract and Introduction

SkillLoop is a campus-oriented platform intended to help students exchange knowledge directly with one another. A student can list skills they are comfortable teaching, skills they want to learn, their availability, and their preferred learning mode. The platform identifies reciprocal learning opportunities and allows a student to send an exchange request.

The project is designed as an academic demonstration of object-oriented programming in C++. The domain model, student search, compatibility calculation, ranking, exchange workflow and CSV persistence are implemented in a C++17 engine. Streamlit provides a browser-based interface, while Python libraries support descriptive analytics.

The architecture deliberately separates presentation from business logic. Streamlit collects user input and displays results; it does not calculate matches. It invokes the C++ executable through a small Python adapter. This makes the C++ engine independently compilable and testable while keeping the demonstration accessible through a web browser.

The current recommendation method is deterministic and rule-based. It is not a machine-learning model and the project does not claim predictive AI.

Keywords: C++, object-oriented programming, Streamlit, reciprocal matching, CSV, data analytics.

02 — NEED AND SCOPE

Problem Statement and Objectives

Problem statement. Students often possess useful skills but lack a structured way to discover classmates who can teach the skills they want to learn. Informal requests are difficult to coordinate, and a one-sided skill overlap does not guarantee a mutually beneficial exchange.

Proposed solution. SkillLoop maintains student skill profiles and searches for pairs where each student can teach at least one skill the other wants to learn. It ranks eligible pairs using availability and learning-mode compatibility, then records exchange requests.

Objectives

• Create and maintain student profiles with skills offered and skills wanted.

• Identify reciprocal skill-exchange opportunities rather than one-sided overlaps.

• Calculate an interpretable compatibility score and rank matches.

• Create, accept and decline exchange requests.

• Persist student and exchange records using portable CSV files.

• Demonstrate major C++ OOP concepts in the actual business-logic implementation.

• Analyse campus skill supply and demand using Python and visualisations.

Scope. The prototype focuses on a small campus community and a demonstrable workflow. Secure authentication, messaging, calendar synchronisation, large-scale concurrency and a production database are outside the current scope.

03 — SYSTEM COMPARISON

Existing System and Proposed System

Existing approach. Students typically rely on class groups, friends or informal social messages to find someone who can teach a skill. These methods are fragmented and do not systematically check whether both participants benefit.

Proposed SkillLoop system. Profiles represent teaching and learning preferences in a consistent structure. The C++ matching engine checks reciprocal compatibility, calculates a score and returns ranked candidates. An exchange request provides a simple workflow for recording the next step.

Aspect	Informal approach	SkillLoop prototype
Discovery	Manual messages and word of mouth	Searchable student profiles and ranked matches
Compatibility	Usually judged informally	Reciprocal skill check plus weighted score
Request tracking	Scattered messages	Pending / Accepted / Declined status
Records	Unstructured	CSV records
Insights	No consolidated view	Supply, demand and skill-gap charts

Expected benefits. More discoverable peer learning opportunities, clearer expectations, a traceable request status, and a simple overview of which skills are commonly offered or requested.

04 — REQUIREMENTS

Requirements and Technology Stack

Functional requirements

• Select a demo student identity and view the community dashboard.

• Create and update a student profile.

• Store teaching skills, learning goals, availability and mode.

• Generate reciprocal matches and show compatibility scores.

• Send exchange requests and allow the recipient to accept or decline.

• Display skill supply, demand and demand-gap analytics.

Non-functional requirements

• Maintain a clear separation between interface and business logic.

• Use portable, human-readable CSV storage for the prototype.

• Provide understandable error messages when the C++ compiler or data files are unavailable.

• Keep the interface responsive and readable in a desktop browser.

• Make the core engine independently compilable and testable.

Technology	Role
C++17 / g++	OOP domain model, matching, search and exchange operations
Streamlit	Browser interface and user interaction
Python subprocess adapter	Calls the compiled C++ engine
Pandas / Matplotlib	Descriptive analysis and charts
CSV	Student and exchange persistence

05 — DESIGN

System Architecture and Data Flow

The design uses a layered architecture. The Streamlit presentation layer communicates with a Python adapter. The adapter compiles the C++ source when required and invokes the engine with structured command-line arguments. C++ reads or updates CSV files and returns tab-separated results for the interface to render.

```
STREAMLIT PRESENTATION → PYTHON ADAPTER → C++17 ENGINE → CSV STORAGE
```

The analytics module independently reads the student CSV file with Pandas. It counts the number of students offering and seeking each skill, calculates a demand gap, and generates charts.

Reason for this design. The C++ engine remains the source of truth for matching and exchange operations. Streamlit keeps the application easy to open through a browser and avoids requiring Qt or a JavaScript frontend. A command-line boundary also allows the C++ engine to be tested without the UI.

Deployment note. The repository includes a Streamlit Community Cloud configuration with `packages.txt` requesting g++. The hosting environment must support installing the compiler and building the C++ executable at startup. A deployed URL must be generated and verified from the team’s own hosting account.

Class Design and Relationships

Class	Responsibility / relationship
User	Abstract base class; private identity data; pure virtual role and dashboardMessage methods
Student	Publicly inherits User; stores course, year, skills, availability and mode
Admin	Publicly inherits User; overrides role-specific virtual behaviour
Match	Encapsulates candidate details and compatibility score; overloads operator>
DataManager	Loads and writes Student and Exchange records in CSV files
StudentService	Implements overloaded searchStudents methods
MatchEngine	Checks reciprocal skill fit, scores and ranks candidates
ExchangeService	Creates exchange requests through DataManager

Inheritance. Student and Admin are specialised forms of User. Shared identity information and the common interface are defined once in the base class.

Association and composition. MatchEngine operates on Student objects supplied by the application. DataManager is used by services to persist domain records. The classes are separated by responsibility rather than putting every operation into one large class.

07 — ACADEMIC MAPPING

Implementation of Major OOP Concepts

OOP concept	Where demonstrated
Encapsulation	Private fields in User and Match; public getters and controlled setters
Abstraction	User is abstract and declares pure virtual role-specific methods
Inheritance	Student and Admin derive from User
Runtime polymorphism	Virtual methods are called through User smart pointers in user_demo
Constructors / destructors	Constructors initialise objects; virtual User destructor supports safe base-pointer destruction
Function overloading	StudentService::searchStudents(name), searchStudents(name, course), searchStudents(name, year)
Operator overloading	Match::operator> compares compatibility scores; ranking uses it
Dynamic memory	std::make_unique creates polymorphic objects with automatic lifetime management

The implementation uses RAII and `std::unique_ptr` rather than manually managing raw pointers. This still demonstrates dynamic allocation while reducing memory-leak risk. The virtual destructor ensures correct destruction through a base-class pointer.

The `user_demo` command provides a small, direct demonstration of runtime polymorphism. It stores Student and Admin objects through the User interface and invokes overridden virtual methods.

08 — CORE LOGIC

Matching Algorithm and Exchange Workflow

Step 1 — Reciprocal eligibility. For students A and B, A must teach at least one skill B wants, and B must teach at least one skill A wants. If either direction is missing, the pair is not returned.

Step 2 — Compatibility score. For an eligible pair, the prototype uses the following transparent heuristic:

```
Score = 0.70 × SkillFit + 0.20 × AvailabilityFit + 0.10 × ModeFit
```

• SkillFit = 100 for a reciprocal pair; non-reciprocal pairs are excluded.

• AvailabilityFit = 100 when availability labels match; otherwise 60.

• ModeFit = 100 when preferred learning modes match; otherwise 70.

For example, a pair with matching availability and mode receives 100. A pair with different availability and mode receives 70 + 12 + 7 = 89. Matches are sorted in descending order using the overloaded Match comparison operator.

Step 3 — Exchange workflow. The requester selects a match and creates a Pending request. The recipient can accept or decline it. The new status is persisted to exchanges.csv.

Algorithm classification. This is rule-based scoring, not machine learning. Future work could learn weights from ratings or successful exchanges, but that is not implemented in this prototype.

09 — IMPLEMENTATION

Interface, Storage and Analytics

Interface pages. Overview presents community metrics and the current student’s skill snapshot. Discover displays reciprocal matches and compatibility. My Profile creates or updates a profile. Exchanges shows requests and recipient actions. Analytics visualises the number of students offering and seeking each skill.

CSV storage. students.csv stores identity, course, year, skills offered, skills wanted, availability and learning mode. exchanges.csv stores request ID, requester, target, skill, status and creation date. CSV is readable and portable, though it is not suitable for concurrent production workloads.

Data analytics. The analytics script transforms semicolon-separated skill lists into long-form observations. It groups by skill and type, counts Offered and Wanted records, and computes Demand gap = Wanted − Offered. It exports a summary CSV and two horizontal bar charts.

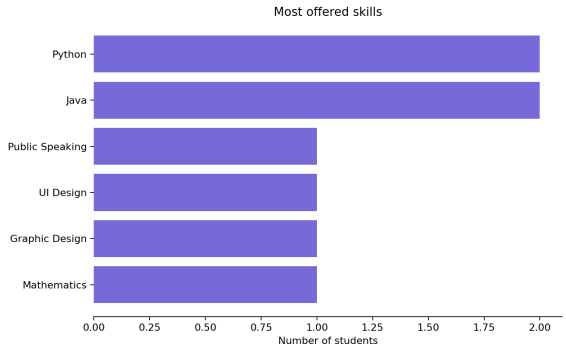


Figure 1. Most offered skills from the included demonstration dataset.

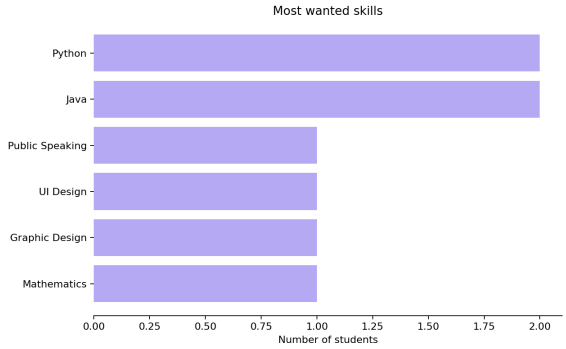


Figure 2. Most wanted skills from the included demonstration dataset.

10 — VERIFICATION

Testing and Results

The C++ engine was compiled with g++ using the C++17 standard. The automated integration test suite runs the executable against an isolated temporary data directory. The following tests passed in the build environment:

Test case	Expected result	Result
Sample data initialisation	Seed student records can be loaded	PASS
Reciprocal match	S001 discovers reciprocal candidate S002	PASS
One-sided overlap	A one-way profile is excluded from reciprocal matches	PASS
Exchange lifecycle	Request can be created and status updated	PASS
Polymorphism demonstration	Student and Admin virtual behaviour is invoked	PASS
Profile update	Existing student record is updated in CSV	PASS

Build verification. The C++ engine compiled successfully and the Python adapter successfully invoked the compiled executable. The analytics script generated the summary CSV and two chart images. Python source files passed syntax compilation.

Verification limitation. The Streamlit runtime could not be launched in the isolated build environment because external package downloads were unavailable. Therefore, the browser UI and cloud deployment still require a final run in an environment with the dependencies installed. Do not claim that a public deployment URL has been tested until the team deploys it.

11 — FINAL SECTION

Conclusion, Contributions and References

Conclusion. SkillLoop demonstrates how object-oriented C++ can power the core logic of a practical peer-learning application. The prototype combines an abstract user hierarchy, encapsulated domain objects, overloaded search functions, operator-based match ranking, CSV persistence and a reciprocal compatibility algorithm. Streamlit makes the workflow accessible through a browser, while Python analytics summarises campus skill supply and demand.

Future scope. Add secure authentication, a relational database, real-time chat, calendar scheduling, notifications, richer availability overlap, user ratings and feedback-based score calibration. These are proposed extensions, not current features.

Individual contributions — confirm before submission.

Member	Proposed individual contribution
Aman — [ROLL NO.]	C++ class hierarchy and OOP implementation; matching engine and reciprocal scoring; C++/Python adapter; in
Iksha — [ROLL NO.]	Streamlit page layout and interactions; profile and exchange workflow; CSV data validation; analytics and char
Both members	Requirements, design review, sample data, integration review, report and viva preparation.

References

• ISO C++ Foundation, C++ documentation: https://isocpp.org/

• Streamlit documentation: https://docs.streamlit.io/

• Python documentation: https://docs.python.org/3/

• Pandas documentation: https://pandas.pydata.org/docs/

• Matplotlib documentation: https://matplotlib.org/stable/

• CMake documentation: https://cmake.org/documentation/

Submission checklist: replace all bracketed identity placeholders; verify the contribution split; add the final GitHub repository and deployed demo URL after testing; attach genuine application screenshots if required by the faculty guide.